

# 엑셀·CSV 전처리와 데이터 시각화 보강

pandas 전처리, 엑셀/CSV 자동화, 그룹화, 시각화를 실무 상황과 연결해 더 자세히 설명합니다.

메인

Chapter 2

세미프로젝트

## 1. Chapter 2의 핵심 관점

Chapter 2는 “데이터를 분석 가능한 형태로 만드는 과정”입니다. 실제 업무 데이터는 바로 분석하기 어렵습니다. 파일이 여러 개로 나뉘어 있거나, 컬럼명이 제각각이거나, 결측치와 이상치가 섞여 있는 경우가 많습니다.

따라서 Chapter 2에서는 단순히 `read_csv()` 나 `plot()` 을 배우는 것이 아니라, 데이터를 읽고 구조를 확인한 뒤, 필요한 형태로 정리하고, 그 결과를 그래프로 설명하는 흐름을 익혀야 합니다.

### 데이터 수집

CSV와 엑셀 파일을 읽습니다.

### 데이터 점검

행/열, 자료형, 결측치를 확인합니다.

### 데이터 정리

필터링, 정렬, 그룹화를 수행합니다.

### 데이터 설명

그래프와 해석 문장으로 결과를 전달합니다.

## 2. pandas를 배우는 이유

pandas는 표 형태 데이터를 다루는 파이썬 라이브러리입니다. 엑셀에서 필터, 정렬, 피벗 테이블로 하던 작업을 코드로 자동화할 수 있습니다. 특히 같은 작업을 반복해야 할 때 pandas의 장점이 커집니다.

### 엑셀 작업과 pandas 작업 비교

| 엑셀 작업 | pandas 작업                                                 | 장점          |
|-------|-----------------------------------------------------------|-------------|
| 파일 열기 | <code>pd.read_csv()</code> , <code>pd.read_excel()</code> | 반복 자동화 가능   |
| 필터    | 조건 필터링                                                    | 조건 재사용 가능   |
| 정렬    | <code>sort_values()</code>                                | 기준 명확화      |
| 피벗    | <code>groupby()</code>                                    | 집계 과정 기록 가능 |

**강의 포인트:** 초보자에게 pandas는 “코드로 하는 엑셀 자동화”라고 설명하면 이해가 빠릅니다.

## 3. 데이터 구조 확인을 자세히 보기

데이터를 읽은 직후 가장 먼저 해야 할 일은 모델링이나 그래프가 아닙니다. 데이터가 어떤 모양인지 확인해야 합니다.

```
df = pd.read_csv("sales.csv")

print(df.head())      # 앞부분 확인
print(df.shape)       # 행과 열 개수 확인
print(df.columns)     # 컬럼명 확인
```

```
print(df.info())      # 자료형과 결측치 확인
print(df.describe())  # 수치형 컬럼의 통계 요약
```

## 각 코드의 해석

- `head()` : 데이터가 제대로 읽혔는지 빠르게 확인합니다.
- `shape` : 데이터 규모를 파악합니다. 행이 너무 적거나 많으면 이유를 확인합니다.
- `columns` : 컬럼명 오타를 방지합니다.
- `info()` : 숫자처럼 보이는 컬럼이 문자열로 들어왔는지 확인합니다.
- `describe()` : 최솟값과 최댓값을 보고 이상치를 의심할 수 있습니다.

## 4. 결측치와 자료형 보강

결측치는 단순히 비어 있는 값이 아니라, 데이터 수집 과정에서 누락되었거나 해당 값이 존재하지 않는 상황을 의미할 수 있습니다. 결측치를 처리할 때는 먼저 왜 비어 있는지 생각해야 합니다.

| 상황        | 처리 예시        | 주의점             |
|-----------|--------------|-----------------|
| 판매 수량 누락  | 0으로 대체 가능    | 정말 판매가 없었는지 확인  |
| 고객 등급 누락  | unknown으로 대체 | 분석 시 별도 그룹으로 해석 |
| 중요 목표값 누락 | 해당 행 제외 검토   | 모델 학습에는 정답값이 필요 |

```
print(df.isna().sum())

df["sales"] = df["sales"].fillna(0)
df["category"] = df["category"].fillna("unknown")
df["date"] = pd.to_datetime(df["date"])
```

## 5. groupby를 실무적으로 이해하기

groupby() 는 Chapter 2에서 가장 중요한 문법 중 하나입니다. 전체 매출보다 더 중요한 것은 “어떤 기준으로 나누어 볼 것인가”입니다. 지점별, 카테고리별, 월별, 회원 등급별로 나누어 보면 의사결정에 필요한 정보가 보입니다.

```
category_summary = df.groupby("category")["sales"].agg(["sum",
"mean", "count"])
print(category_summary)
```

### 결과 해석 예시

커피 카테고리의 총매출이 가장 높다면 핵심 매출원은 커피라고 볼 수 있습니다. 하지만 평균 매출은 푸드가 더 높다면, 푸드 메뉴는 거래 수는 적어도 객단가가 높은 상품일 수 있습니다. 이처럼 집계 결과는 하나의 숫자만 보지 말고 합계, 평균, 건수를 함께 봐야 합니다.

## 6. 시각화 보강

그래프는 예쁘게 그리는 것이 목적이 아닙니다. 숫자로만 보기 어려운 패턴을 빠르게 파악하고, 다른 사람에게 분석 결과를 설명하기 위한 도구입니다.

| 그래프   | 적합한 질문           | 예시         |
|-------|------------------|------------|
| 선 그래프 | 시간에 따라 어떻게 변했는가? | 월별 매출 추이   |
| 막대그래프 | 어떤 그룹이 더 큰가?     | 카테고리별 총매출  |
| 산점도   | 두 변수는 관계가 있는가?   | 광고비와 매출    |
| 히스토그램 | 값이 어디에 몰려 있는가?   | 고객 구매금액 분포 |

## 그래프 해석 문장 틀

그래프에서 [관찰한 현상]이 보입니다.  
이는 [가능한 원인 또는 의미]로 해석할 수 있습니다.  
따라서 [후속 분석 또는 실무 행동]을 검토할 수 있습니다.

## 7. 자주 나는 오류와 해결법

| 오류                | 원인        | 점검 방법                                          |
|-------------------|-----------|------------------------------------------------|
| FileNotFoundError | 파일 위치 오류  | 현재 폴더와 파일명을 확인                                 |
| KeyError          | 컬럼명 오타    | <code>df.columns</code> 출력                     |
| ValueError        | 자료형 변환 실패 | 문자, 쉼표, 결측치 확인                                 |
| 한글 깨짐             | 인코딩 문제    | <code>utf-8-sig</code> , <code>cp949</code> 시도 |

## 8. Chapter 2 확장 실습 전략

1. CSV 파일 하나를 읽고 구조를 확인합니다.
2. 동일한 구조의 CSV 파일 3개를 만들어 자동으로 합칩니다.
3. 결측치를 일부러 추가한 뒤 처리해 봅니다.
4. 카테고리별, 지점별, 날짜별로 각각 집계합니다.
5. 각 집계 결과에 맞는 그래프를 선택합니다.
6. 그래프마다 해석 문장 2개를 작성합니다.
7. 최종 결과를 CSV와 txt 리포트로 저장합니다.

**Chapter 2 핵심 정리:** 전처리는 분석의 준비 단계가 아니라 분석 품질을 결정하는 핵심 단계입니다. 데이터를 읽고, 확인하고, 정리하고, 시각화하고, 해석하는 흐름을 반복해서 익히는 것이 중요합니다.

[← Chapter 2 목록](#)

[세미프로젝트로 이동 →](#)